CSA04-Operating Systems

TASK 10 – Measure and Compare

V. Reddy Sreya Regno: 192525124

1. Introduction

The performance of programs in Linux can be measured using system calls, execution time, process creation time and thread creation time.

Linux provides tools such as strace and time to analyse system performance.

2. Number of System Calls Using strace -c

The strace -c command counts the system calls made by a program.

Objective: Measure how different buffer sizes affect the number of read() and write() system calls during file copying.

Commands

First, compile your copy program:

gcc copy.c -o copy

Run the program with different buffer sizes (assuming your copy program accepts buffer size as its third argument):

strace -c ./copy input.txt output.txt 1024
strace -c ./copy input.txt output.txt 4096
strace -c ./copy input.txt output.txt 16384
strace -c ./copy input.txt output.txt 65536

Expected Observation

The following is an illustrative result, not an actual measurement.

Buffer Size	Approx. read() calls	Approx. write() calls
1 KB	1024	1024
4 KB	256	256
16 KB	64	64
64 KB	16	16

Assumption: Input file size is 1 MB, with no partial writes or extra data operations.

Evaluation:

- Small buffers require more system calls.
- Large buffers reduce the number of read and write operations.
- Fewer system calls reduce kernel-entry overhead.
- Therefore, larger buffers generally improve file-copy performance.

3. Measuring User Mode and Kernel Mode Time

The time command measures the execution time of a program.

Command:

time ./copy input.txt output.txt 4096

Example Output

real 0m0.025s
user 0m0.005s
sys 0m0.012s

Time	Meaning
Real	Total elapsed time
User	Time spent executing user-mode code
Sys	Time spent executing kernel-mode code

Evaluation:

- User time measures application-level CPU work.
- System time measures operating-system work.
- Real time includes execution and waiting time.
- A large system time may indicate frequent system calls or I/O operations.

4. Time to Create 1,000 Processes and 1,000 Threads

To compare process and thread creation, we can write a C program using fork() and pthread_create().

C Program: Process and Thread Comparison

Save as compare.c.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
```

```
#include <pthread.h>
#include <time.h>

#define N 1000

double get_time()
{
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec + t.tv_nsec / 1000000000.0;
}

void *thread_function(void *arg)
{
    return NULL;
}

int main()
{
    pid_t pid;
    pthread_t threads[N];
    int i;
    double start, end;

    // Measure process creation
    start = get_time();

    for (i = 0; i < N; i++)
    {
        pid = fork();

        if (pid == 0)
            _exit(0);

        if (pid < 0)
        {
            perror("fork");
            return 1;
        }
    }

    for (i = 0; i < N; i++)
        wait(NULL);
```

```
    end = get_time();

    printf("Time for 1000 processes: %.6f seconds\n",
           end - start);

    // Measure thread creation
    start = get_time();

    for (i = 0; i < N; i++)
    {
        if (pthread_create(&threads[i], NULL,
                           thread_function, NULL) != 0)
        {
            printf("Thread creation failed\n");
            return 1;
        }
    }

    for (i = 0; i < N; i++)
        pthread_join(threads[i], NULL);

    end = get_time();

    printf("Time for 1000 threads: %.6f seconds\n",
           end - start);

    return 0;
}
```

Compilation and Execution

Step 1: Compile

```
gcc compare.c -o compare -pthread
```

Step 2: Run

```
./compare
```

Example Output

Illustrative values only
Time for 1000 processes: 0.850000 seconds
Time for 1000 threads: 0.065000 seconds

Actual values depend on the processor, RAM, Linux version and system load. Run the program on your Ubuntu system and record your own results.

Comparison Table

Factor	Processes	Threads
Creation	Slower	Faster
Memory usage	Higher	Lower
Address space	Separate	Shared
Communication	IPC required	Shared variables
Isolation	High	Low
Management overhead	Higher	Lower

Evaluation:

- Creating 1,000 processes requires more operating-system resources.
- Creating 1,000 threads is generally faster.
- Processes have separate virtual address spaces.
- Threads share code, data and heap memory within the same process.
- Each thread still needs its own stack and execution state.

5. Alternative Concept and Complexity Analysis – Threads vs Processes

If threads are created instead of processes, the system generally uses less memory and takes less time to create them.

Why are threads faster?

Processes:

- Each process has a separate address space.
- The OS maintains separate process information and resources.
- Process creation requires more memory management.

Threads:

- Threads share the same code, data and heap.
- Each thread has its own stack and CPU registers.
- Less memory and fewer resources are required for creation.

Memory Usage Comparison

Memory Component	Processes	Threads
Code	Separate address spaces (pages may be shared)	Shared
Data	Separate	Shared
Heap	Separate	Shared
Stack	Separate for each process	Separate for each thread
CPU registers	Separate	Separate

Complexity Analysis

Operation	Time Complexity
Creating one process	OS-dependent
Creating 1,000 processes	Approximately O(1000)
Creating one thread	OS-dependent
Creating 1,000 threads	Approximately O(1000)

Both operations have linear total creation work, but thread creation generally has a lower constant overhead.

6. Overall Measurement and Evaluation

Measurement	Expected Observation	Conclusion
Small buffer size	More read/write calls	Higher overhead
Large buffer size	Fewer read/write calls	Better efficiency
User time	Application processing	Measures user-mode work
Kernel time	System-call processing	Measures kernel-mode work
Process creation	More time and memory	Higher overhead
Thread creation	Less time and memory	Lower overhead

How measurements support earlier judgements:

1. strace -c shows that larger buffers generally reduce system-call frequency.
2. The time command identifies the time spent in user and kernel modes.
3. Process creation measurements show the overhead of maintaining separate address spaces.
4. Thread measurements generally show lower creation overhead because memory is shared.
5. The results support the conclusion that buffer optimization and threads can improve performance for suitable workloads.

7. Conclusion

1. strace -c helps measure the number of system calls.
2. Larger buffers reduce read and write operations.
3. The time command measures user time, system time and real time.

4. Creating 1,000 threads is generally faster than creating 1,000 processes.
5. Processes provide better isolation, whereas threads provide efficient memory sharing.

Result: Linux performance measurement tools help evaluate system calls, execution time, process creation and thread creation. The comparison demonstrates the importance of efficient resource management in operating systems.